

Bài 8: Mảng, con trỏ và chuỗi ký tự

Bài giảng LẬP TRÌNH CƠ BẢN

Tài liệu tham khảo

- ▶ Kỹ thuật lập trình C: cơ sở và nâng cao, Phạm Văn Ất, Nhà xuất bản KHKT – Chương 6
- ▶ The C programming language 2nd Edition, Brian Kernighan and Dennis Ritchie, Prentice Hall Software Series – Chương 4

Mục tiêu của bài học

- ▶ Các phần tử của mảng và các chỉ số
- ▶ Khai báo mảng
- ▶ Cách quản lý mảng trong C
- ▶ Cách khởi tạo mảng
- ▶ Tìm hiểu chuỗi / mảng ký tự
- ▶ Tìm hiểu mảng hai chiều
- ▶ Cách khởi tạo mảng hai chiều

Mục tiêu bài học

- Tìm hiểu về con trỏ và khi nào thì sử dụng con trỏ
- Cách sử dụng biến con trỏ và các toán tử con trỏ
- Gán giá trị cho con trỏ
- Phép toán trên con trỏ
- So sánh con trỏ
- Con trỏ và mảng một chiều
- Con trỏ và mảng nhiều chiều
- Tìm hiểu cách cấp phát bộ nhớ

Mục tiêu bài học

- Giải thích biến và hằng chuỗi.
- Giải thích con trỏ đến chuỗi.
- Thực hiện các thao tác nhập/xuất chuỗi.
- Giải thích các hàm thao tác chuỗi.

Các phần tử và chỉ số của mảng

- ▶ Mảng là một tập hợp các phần tử cố định có cùng một kiểu, gọi là kiểu phần tử.
 - ▶ Kiểu phần tử có thể là có các kiểu bất kỳ: ký tự, số, chuỗi ký tự...;
 - ▶ cũng có khi ta sử dụng kiểu mảng để làm kiểu phần tử cho một mảng (trong trường hợp này ta gọi là mảng của mảng hay mảng nhiều chiều)
- ▶ Mỗi phần tử được xác định bằng một số thứ tự (còn gọi là chỉ số) duy nhất trong mảng
- ▶ Số chiều của mảng được xác định bằng số các chỉ số cần thiết để định danh duy nhất từng phần tử
- ▶ Chỉ số là một số nguyên không âm trong [] đặt ngay sau tên mảng
- ▶ Chỉ số của mảng (trong C) được bắt đầu là 0
- ▶ Mảng *player* với 11 phần tử :

player[0], *player*[1], *player*[2],.... *player*[10]

Khai báo mảng

- Các đặc tính riêng của mảng cần được định nghĩa.

Lớp lưu trữ

Kiểu dữ liệu của các phần tử

Tên mảng đại diện cho vị trí phần tử đầu tiên

Kích thước mảng một hằng số

Khai báo mảng (tt.)

Khai báo mảng giống như cách khai báo biến. Chỉ khác là tên mảng được theo sau bởi một hoặc nhiều biểu thức đặt trong cặp dấu ngoặc vuông [], để xác định kích thước của mảng.

```
int player[11];
```


Các qui tắc

- ▶ Các phần tử của mảng có cùng kiểu dữ liệu
- ▶ Mỗi phần tử của mảng có thể được sử dụng như một biến riêng lẻ
- ▶ Kiểu dữ liệu của mảng có thể là `int`, `char`, `float` hoặc `double`

Quản lý mảng trong C

- ▶ Trong ngôn ngữ C, mảng được “đổi xử” không giống hoàn toàn với biến
- ▶ Hai mảng có cùng kiểu và cùng kích thước cũng không được xem là tương đương nhau
- ▶ Không thể gán trực tiếp một mảng cho một mảng khác.
- ▶ Không thể gán trị cho toàn bộ mảng, mà phải gán trị cho từng phần tử của mảng

Quản lý mảng trong C (tt)

```
/*Input values are accepted from the user  
into the array ary[10]*/
```

```
#include <stdio.h>  
void main() {  
    int ary[10];  
    int i, total, high, n;  
    printf("Nhap so ptu cua mang <10\n");  
    scanf("%d", &n);  
    for(i=0; i<n; i++) {  
        printf("\n Enter value: %d : ", i+1);  
        scanf("%d", &ary[i]);  
    }
```

Quản lý mảng trong C (tt)

```
/* Displays highest of the entered values */
    high = ary[0];
    for(i=1; i<10; i++){
        if(ary[i] > high) high = ary[i];
    }
    printf("\nHighest value entered was %d", high);

/*prints average of values entered for ary[10] */
    for(i=0,total=0; i<10; i++) total = total +
ary[i];
    printf("\nThe average of the elements of ary
is%d",total/i);
}
```

Khởi tạo mảng

- Mỗi phần tử của một mảng auto cần được khởi tạo riêng rẽ.
- Trong ví dụ sau các phần tử của mảng được gán giá trị bằng cách sử dụng vòng lặp **for**

```
#include <stdio.h>

void main() {
    char alpha[26];
    int i, j;
    for(i=65, j=0; i<91; i++, j++) {
        alpha[j] = i;
        printf("The character now assigned is%c\n", alpha[j]);
    }
    getchar();
}
```

Khởi tạo mảng (tt)

- Khởi tạo khi khai báo

```
int deci[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

```
static float rates[4] = {0.0, -2.5, 13.75, 18.0};
```

```
char company[5] = {'A', 'P', 'P', 'L', 'E'};
```

```
int marks[100] = {15, 13, 11, 9};
```

- Trong trường hợp mảng extern và static, các phần tử được tự động khởi tạo với giá trị 0

```
int ary[] = {1, 2, 3, 4, 5};
```

Mảng hai chiều

- ▶ Mảng đa chiều đơn giản nhất và thường được dùng nhất là mảng hai chiều
- ▶ Mảng hai chiều có thể xem như là một mảng với mỗi phần tử là mảng một chiều
- ▶ Về logic, một mảng hai chiều trông giống như một bảng lịch trình xe lửa, gồm các dòng và các cột
- ▶ Khai báo mảng hai chiều:
`int temp[4][3];`

Khởi tạo mảng đa chiều

```
int ary[3][4]  
    = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
```

Kết quả của phép gán trên như sau:

ary[0][0] = 1	ary[0][1] = 2	ary[0][2] = 3	ary[0][3] = 4
ary[1][0] = 5	ary[1][1] = 6	ary[1][2] = 7	ary[1][3] = 8
ary[2][0] = 9	ary[2][1] = 10	ary[2][2] = 11	ary[2][3] = 12

Khởi tạo mảng đa chiều (tt)

```
int ary[3][4] ={{1,2,3},{4,5,6},{7,8,3}};
```

Kết quả của phép gán trên như sau:

<code>ary[0][0] =1</code>	<code>ary[0][1]=2</code>	<code>ary[0][2]=3</code>	<code>ary[0][3]=0</code>
<code>ary[1][0]=4</code>	<code>ary[1][1]=5</code>	<code>ary[1][2]=6</code>	<code>ary[1][3]=0</code>
<code>ary[2][0]=7</code>	<code>ary[2][1]=8</code>	<code>ary[2][2]=3</code>	<code>ary[2][3]=0</code>

Khởi tạo mảng đa chiều (tt)

Một mảng chuỗi hai chiều được khai báo theo cách sau:

```
char str_ary[25][80];
```

Ví dụ

/* Chương trình nhập các số vào một mảng hai chiều. */

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int arr[2][3];
```

```
    int row, col;
```

```
    for(row = 0; row < 2; row++)
```

```
    {
```

```
        for(col = 0; col < 3; col++)
```

```
        {
```

```
            printf("\nEnter a Number at [%d][%d]: ", row, col);
```

```
            scanf("%d", &arr[row][col]);
```

```
        }
```

```
    }
```

```
for(row = 0; row < 2; row++)
{
    for(col = 0; col < 3; col++)
    {
        printf("\nThe Number at [%d][%d] is %d",
row, col, arr[row][col]);
    }
}
```

Mảng hai chiều - Ví dụ

```
#include <stdio.h>
#include <string.h>
void main (){
    int i, n = 0;
    int item;
    char x[10][12];
    char temp[12];
    clrscr();
    printf("Enter each string on a separate line\n\n");
    printf("Type 'END' when over \n\n");
    /* read in the list of strings */
    do{
        printf("String %d : ", n+1);
        scanf("%s", x[n]);
    } while (strcmp(x[n++], "END"));
    /*reorder the list of strings */
```

Mảng hai chiều - Ví dụ (tt.)

```
n = n - 1;
for(item=0; item<n-1; ++item) {
    /* find lowest of remaining strings */
    for(i=item+1; i<n; ++i) {
        if(strcmp (x[item], x[i]) > 0){
            /*interchange two stings */
            strcpy (temp, x[item]);
            strcpy (x[item], x[i]);
            strcpy (x[i], temp);
        }
    }
}

/* Display the arranged list of strings */
printf("Recorded list of strings : \n");
for(i = 0; i < n ; ++i) {
    printf("\nString %d is %s", i+1, x[i]);
}
}
```

Con trỏ là gì?

- Con trỏ là một biến, nó chứa địa chỉ ô nhớ của một biến khác
- Nếu một biến chứa địa chỉ của một biến khác, thì biến này được gọi là con trỏ *trỏ đến* biến thứ hai
- Con trỏ cung cấp phương thức truy xuất gián tiếp đến giá trị của một phần tử dữ liệu
- Các con trỏ có thể trỏ đến các biến có kiểu dữ liệu cơ bản như `int`, `char`, `double`, hay dữ liệu tập hợp như mảng hoặc cấu trúc.

Con trỏ được sử dụng để làm gì?

Các tình huống con trỏ có thể được sử dụng:

- Để trả về nhiều hơn một giá trị từ một hàm
- Để truyền mảng và chuỗi từ một hàm đến một hàm khác thuận tiện hơn
- Để làm việc với các phần tử của mảng thay vì truy xuất trực tiếp vào các phần tử này
- Để cấp phát bộ nhớ và truy xuất bộ nhớ (Cấp phát bộ nhớ trực tiếp)

Biến con trỏ

- Khai báo con trỏ: chỉ ra một kiểu cơ sở và một tên biến được đặt trước bởi dấu *

Cú pháp khai báo tổng quát:

```
type *name;
```

Ví dụ:

```
int *var2;
```


Các toán tử con trỏ

- Hai toán tử đặc biệt được sử dụng với con trỏ:

& và *****

- **&** là toán tử một ngôi và nó trả về địa chỉ ô nhớ của toán hạng

var2 = &var1;

- Toán tử ***** là phần bổ xung của toán tử **&**. Đây là toán tử một ngôi và nó trả về giá trị chứa trong vùng nhớ được trỏ đến bởi biến con trỏ

temp = *var2;

Gán trị đối với con trỏ

- Các giá trị có thể được gán cho con trỏ thông qua toán tử **&**.

ptr_var = &var;

- Ở đây địa chỉ của var được lưu vào biến ptr_var.
- Cũng có thể gán giá trị cho con trỏ thông qua một biến con trỏ khác trỏ có cùng kiểu.

ptr_var = &var;

ptr_var2 = ptr_var;

Gán trị đối với con trỏ (tt)

- Có thể gán giá trị cho các biến thông qua con trỏ

***ptr_var = 10;**

- Câu lệnh trên gán giá trị 10 cho biến var nếu ptr_var đang trỏ đến var

Phép toán con trỏ

- Chỉ có thể thực hiện phép toán cộng và trừ trên con trỏ

```
int var, * ptr_var;
```

```
ptr_var = & var;
```

```
var = 500;
```

```
ptr_var ++;
```

- Giả sử biến **var** được lưu trữ tại địa chỉ **1000**
- `ptr_var` lưu giá trị 1000. Vì số nguyên có kích thước là 2 bytes, nên sau biểu thức “`ptr_var++;`” `ptr_var` sẽ có giá trị là 1002 mà không là 1001

Phép toán con trỏ (tt)

<code>++ptr_var or ptr_var++</code>	Trỏ đến số nguyên kế tiếp đứng sau var
<code>--ptr_var or ptr_var--</code>	Trỏ đến số nguyên đứng trước var
<code>ptr_var + i</code>	Trỏ đến số nguyên thứ i sau var
<code>ptr_var - i</code>	Trỏ đến số nguyên thứ i trước var
<code>++*ptr_var or (*ptr_var)++</code>	Sẽ tăng trị var bởi 1
<code>*ptr_var++</code>	Sẽ tác động đến giá trị của số nguyên kế tiếp sau var

Phép toán con trỏ (tt)

- Mỗi lần con trỏ được tăng trị, nó trỏ đến ô nhớ của phần tử kế tiếp
- Mỗi lần con trỏ được giảm trị, nó trỏ đến ô nhớ của phần tử đứng trước nó
- Tất cả con trỏ sẽ tăng hoặc giảm trị theo kích thước của kiểu dữ liệu mà chúng đang trỏ đến

So sánh con trỏ

- Hai con trỏ có thể được so sánh trong một biểu thức quan hệ nếu chúng trỏ đến các biến có cùng kiểu dữ liệu
- Giả sử ptr_a và ptr_b là hai biến con trỏ trỏ đến các phần tử dữ liệu a và b. Trong trường hợp này, các phép so sánh sau là có thể:

So sánh con trỏ (tt)

<code>ptr_a < ptr_b</code>	Trả về giá trị true nếu a được lưu trữ ở vị trí trước b
<code>ptr_a > ptr_b</code>	Trả về giá trị true nếu a được lưu trữ ở vị trí sau b
<code>ptr_a <= ptr_b</code>	Trả về giá trị true nếu a được lưu trữ ở vị trí trước b hoặc ptr_a và ptr_b trỏ đến cùng một vị trí
<code>ptr_a >= ptr_b</code>	Trả về giá trị true nếu a được lưu trữ ở vị trí sau b hoặc ptr_a và ptr_b trỏ đến cùng một vị trí
<code>ptr_a == ptr_b</code>	Trả về giá trị true nếu cả hai con trỏ ptr_a và ptr_b trỏ đến cùng một phần tử dữ liệu.
<code>ptr_a != ptr_b</code>	Trả về giá trị true nếu cả hai con trỏ ptr_a và ptr_b trỏ đến các phần tử dữ liệu khác nhau nhưng có cùng kiểu dữ liệu.
<code>ptr_a == NULL</code>	Trả về giá trị true nếu ptr_a được gán giá trị NULL (0)

Con trỏ và mảng một chiều

- Địa chỉ của một phần tử mảng có thể được biểu diễn theo hai cách:
 - Sử dụng ký hiệu & trước một phần tử mảng.
 - Sử dụng một biểu thức trong đó chỉ số của phần tử được cộng vào tên của mảng.

Con trỏ và mảng một chiều-Ví dụ

```
#include<stdio.h>
void main() {
    static int ary[10]
        = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int i;
    for (i= 0; i<10; i++) {
        printf("\ni=%d, ary[i]=%d, * (ary+i)=%d",
            i, ary[i], * (ary + i));
        printf("&ary[i]=%X, ary+i=%X", &ary[i],
            ary+i);
        /*%X gives unsigned hexadecimal*/
    }
}
```

Con trỏ và mảng một chiều-ví dụ tt

i=0	ary[i]=1	*(ary+i)=1	&ary[i]=194	ary+i = 194
i=1	ary[i]=2	*(ary+i)=2	&ary[i]=196	ary+i = 196
i=2	ary[i]=3	*(ary+i)=3	&ary[i]=198	ary+i = 198
i=3	ary[i]=4	*(ary+i)=4	&ary[i]=19A	ary+i = 19A
i=4	ary[i]=5	*(ary+i)=5	&ary[i]=19C	ary+i = 19C
i=5	ary[i]=6	*(ary+i)=6	&ary[i]=19E	ary+i = 19E
i=6	ary[i]=7	*(ary+i)=7	&ary[i]=1A0	ary+i = 1A0
i=7	ary[i]=8	*(ary+i)=8	&ary[i]=1A2	ary+i = 1A2
i=8	ary[i]=9	*(ary+i)=9	&ary[i]=1A4	ary+i = 1A4
i=9	ary[i]=10	*(ary+i)=10	&ary[i]=1A6	ary+i = 1A6

Con trỏ và mảng đa chiều

- Mảng hai chiều có thể được định nghĩa như là một con trỏ trỏ tới một nhóm các mảng một chiều liên tiếp nhau

- Khai báo một mảng hai chiều có thể như sau:

`data_type (*ptr_var) [expr 2];`

thay vì

`data_type var [expr1] [expr 2];`

Con trỏ và chuỗi

```
#include <stdio.h>
#include <string.h>
void main () {
    char a, str[81], *ptr;
    printf("\nEnter a sentence:");
    gets(str);
    printf("\nEnter character to search for:");
    a = getche();
    ptr = strchr(str,a);
    /* return pointer to char*/
    printf( "\nString starts at address: %u",str);
    printf("\nFirst occurrence of the character is at address: %u ",ptr);
    printf("\n Position of first occurrence(starting from 0)is: % d",
ptr_str);
}
```

Con trỏ và chuỗi (tt)

Enter a sentence: *We all live in a yellow submarine*

Enter character to search for: Y

String starts at address: 65420.

First occurrence of the character is at address: 65437.

Position of first occurrence (starting from 0) is: 17

Cấp phát bộ nhớ

Hàm **malloc()** là một trong các hàm được sử dụng thường xuyên nhất để thực hiện việc cấp phát bộ nhớ từ vùng nhớ còn tự do.

Tham số của hàm **malloc()** là một số nguyên xác định số bytes cần cấp phát.

Cấp phát bộ nhớ (tt)

```
#include<stdio.h>
#include<malloc.h>
void main()
{
    int *p,n,i,j,temp;
    printf("\n Enter number of elements in the array :");
    scanf("%d",&n);
    p=(int*)malloc(n*sizeof(int));
    for(i=0;i<n;++i) {
        printf("\nEnter element no. %d:",i+1);
        scanf("%d",p+i); }
    for(i=0;i<n-1;++i)
        for(j=i+1;j<n;++j)
            if(*(p+i)>*(p+j)) {
                temp=*(p+i);
                *(p+i)=*(p+j);
                *(p+j)=temp; }
    for(i=0;i<n;++i)
        printf("%d\n",*(p+i));
}
```



Hàm `free()`

Hàm `free()` được sử dụng để giải phóng bộ nhớ khi nó không cần dùng nữa.

Cú pháp:

```
void free(void*ptr);
```

Hàm này giải phóng không gian được trả bởi *ptr*, để dùng cho tương lai.

ptr phải được dùng trước đó với lời gọi hàm `malloc()`, `calloc()`, hoặc `realloc()`.

Hàm free() - tt

```
#include <stdio.h>
#include <stdlib.h>
/*required for the malloc and free functions*/
int main() {
    int number;
    int *ptr;
    int i;
    printf("How many ints would you like store? ");
    scanf("%d", &number);
    ptr = (int *) malloc (number*sizeof(int));
    /*allocate memory */
    if(ptr!=NULL) {
        for(i=0 ; i<number ; i++){
            *(ptr+i) = i;
        }
    }
}
```

Còn tiếp...

Hàm free() - tt

```
    for(i=number ; i>0 ; i--) {
        printf("%d\n",*(ptr+(i-1)));
        /* print out in reverse order */
    }
    free(ptr); /* free allocated memory */
    return 0;
}
else {
    printf("\nMemory allocation failed -
    not enough memory.\n");
    return 1;
}
}
```

Hàm calloc()

calloc tương tự như **malloc**, nhưng điểm khác biệt chính là mặc nhiên giá trị 0 được lưu vào không gian bộ nhớ vừa cấp phát

calloc yêu cầu hai tham số

- Tham số thứ nhất là số lượng các biến cần cấp phát bộ nhớ
- Tham số thứ hai là kích thước của mỗi biến

Cú pháp:

```
void *calloc( size_t num, size_t size );
```

Hàm calloc() - tt

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    float *calloc1, *calloc2;
    int i;
    calloc1 = (float *)calloc(3, sizeof(float));
    calloc2 = (float *)calloc(3, sizeof(float));
    if(calloc1!=NULL && calloc2!=NULL) {
        for(i=0 ; i<3 ; i++) {
            printf("calloc1[%d] holds %05.5f ", i,
                calloc1[i]);
            printf("\ncalloc2[%d] holds %05.5f",
                i, *(calloc2+i));
        }
    }
```

Còn tiếp.....

Hàm calloc() - tt

```
    free(calloc1);  
    free(calloc2);  
    return 0;  
}  
else{  
    printf("Not enough memory\n");  
    return 1;  
}  
}
```

Hàm `realloc()`

Có thể cấp phát lại cho một vùng đã được cấp (thêm/bớt số bytes) bằng cách sử dụng hàm **`realloc`**, mà không làm mất dữ liệu.

`realloc` nhận hai tham số

- Tham số thứ nhất là con trỏ tham chiếu đến bộ nhớ
- Tham số thứ hai là tổng số byte muốn cấp phát
- Cú pháp:

```
void *realloc( void *ptr, size_t size );
```

Hàm realloc() - tt

```
#include<stdio.h>
#include <stdlib.h>
int main() {
    int *ptr;
    int i;
    ptr = (int *)calloc(5, sizeof(int *));
    if(ptr!=NULL) {
        *ptr = 1;  *(ptr+1) = 2;
        ptr[2] = 4;  ptr[3] = 8;  ptr[4] = 16;
        ptr = (int *)realloc(ptr, 7*sizeof(int));
        if(ptr!=NULL) {
            printf("Now allocating more memory...\n");
            ptr[5] = 32;  /* now it's legal! */
            ptr[6] = 64;
        }
    }
}
```


Hàm realloc() - tt

```
    for(i=0;i<7;i++) {
        printf("ptr[%d] holds %d\n", i, ptr[i]);
    }
    realloc(ptr,0);
    /* same as free(ptr); - just fancier! */
    return 0;
}
else {
    printf("Not enough memory-realloc failed.\n");
    return 1;
}
}
else {
    printf("Not enough memory-calloc failed.\n");
    return 1;
}
}
```

Các Biến Chuỗi

- Chuỗi là mảng ký tự kết thúc bởi ký tự `null` (`'\0'`).
- Có thể gán các hằng chuỗi cho các biến chuỗi.
- Hằng chuỗi là một chuỗi các ký tự nằm trong dấu nháy kép.
- Ký tự `null '\0'` được tự động thêm vào biểu diễn bên trong của chuỗi.
- Khi khai báo một biến chuỗi, hãy dành thêm một phần tử trống cho ký tự kết thúc.

Khai Báo Biến Chuỗi

- Khai báo một biến chuỗi tiêu biểu:

char str[10];

- **str** là một biến mảng ký tự có thể lưu giữ tối đa 10 ký tự bao gồm cả ký tự kết thúc.

Các thao tác Nhập/Xuất chuỗi

- Sử dụng các hàm trong thư viện nhập/xuất chuẩn `stdio.h` để thực hiện các thao tác nhập/xuất chuỗi.
- Hàm `gets()` là cách đơn giản nhất để nhập vào một chuỗi thông qua thiết bị nhập chuẩn.
- Các ký tự được nhập vào cho đến khi ấn phím Enter
- Hàm `gets()` thay thế ký tự sang dòng mới `'\n'` bằng ký tự `'\0'`
- Cú pháp: **`gets(str);`**

Các thao tác Nhập/Xuất chuỗi - tt

- Hàm puts() được dùng để hiển thị một chuỗi trên thiết bị xuất chuẩn.
- Cú pháp : **puts(str);**
- Các hàm scanf() và printf() được sử dụng để nhập và hiển thị các kiểu dữ liệu hỗn hợp trong cùng một câu lệnh.
- Cú pháp để nhập chuỗi:

scanf("%s", str);

- Cú pháp để hiển thị chuỗi:

printf("%s", str);

Các hàm về chuỗi

Các hàm xử lý chuỗi nằm trong tập tin **string.h**.

Một số thao tác được thực hiện bởi các hàm này là:

- Ghép chuỗi
- So sánh chuỗi
- Xác định vị trí một ký tự trong chuỗi
- Sao chép một chuỗi sang chuỗi khác
- Tính chiều dài chuỗi

Hàm strcat()

- Nối hai giá trị chuỗi vào một chuỗi.
- Cú pháp:

strcat(str1, str2);

- Nối str2 vào cuối chuỗi str1
- Trả về str1

Hàm strcmp()

- So sánh hai chuỗi và trả về một giá trị số nguyên dựa trên kết quả của sự so sánh.
- Cú pháp:

strcmp(str1, str2);

- Hàm trả về một giá trị:
 - Nhỏ hơn 0, nếu $\text{str1} < \text{str2}$
 - 0, nếu str1 giống str2
 - Lớn hơn 0, nếu $\text{str1} > \text{str2}$

Hàm strchr()

- Xác định vị trí xuất hiện của một ký tự trong một chuỗi.
- Cú pháp: **strchr(str, chr);**
- Hàm trả về :
 - con trỏ trỏ đến vị trí tìm được đầu tiên của ký tự (trỏ bởi **chr**) trong chuỗi **str**.
 - NULL nếu **chr** không có trong chuỗi

Hàm strcpy()

- Sao chép giá trị trong một chuỗi vào một chuỗi khác.
- Cú pháp:

strcpy(str1, str2);

- Giá trị của str2 được chép vào str1
- Hàm trả về **str1**

Hàm strlen()

- Xác định chiều dài của chuỗi.
- Cú pháp:

strlen(str);

- Hàm trả về một giá trị nguyên là độ dài của **str**.

Tóm tắt nội dung

- ▶ Mảng một chiều và nhiều chiều
- ▶ Con trỏ và địa chỉ
- ▶ Các phép toán với con trỏ
- ▶ Liên hệ giữa con trỏ và mảng
- ▶ Xâu ký tự và một số hàm làm việc với xâu ký tự

Thảo luận

- ▶ phép gán con trỏ, truy xuất giá trị qua con trỏ, số học địa chỉ.
- ▶ truy xuất vượt chỉ số và cách xử lý trong C.

CÂU HỎI VÀ BÀI TẬP

- ▶ Bài 14: Nhập 2 mảng $A(n,m)$, $B(m,n)$ phần tử kiểu số thực, tính và in mảng $C=A*B$
- ▶ Bài 15: Nhập 2 mảng $A(n,m)$, $B(m,n)$ phần tử kiểu số thực, kiểm tra A có là hoán vị của B hay không
- ▶ Bài 16: Nhập $A(n,n)$ với n không giới hạn trước, kiểm tra A có là ma trận đơn vị không?
- ▶ Bài 17: Xây dựng ma trận $A(n,m)$, sao cho các phần tử có giá trị theo dạng xoắn ốc (n, m không giới hạn trước)
- ▶ Bài 18: Nhập chuỗi họ tên (không quá 40 ký tự), chuẩn hoá chuỗi đó (ký tự đầu từ viết hoa, các ký tự khác viết thường, các từ cách nhau 1 dấu cách)
- ▶ Bài 19: Nhập 3 chuỗi s1, s2, s3 (không quá 40 ký tự), thay chuỗi s2 bằng s3 trong s1
- ▶ Bài 20: Nhập chuỗi ký tự. Đưa chuỗi đó về dạng chuẩn
- ▶ Bài 21: Nhập chuỗi ký tự và tìm từ dài nhất trong chuỗi.

HỎI VÀ ĐÁP